Programozás II.

1. labor

Szekeres György

2026. február 22.

Tartalom

1. A félév beosztása
2. Bevezetés
3. Első program
4. Fontos információk
5. Kiírás és bekérés
6. Azonosítók
7. Változók
8. Változók típusai
9. Fordítás idejű inicializálás
10. Futás idejű inicializálás
11. Változók maximum és minimum értékei
12. Konvertálás, castolás
13. Parse, TryParse
14. Elágazások
15. Ciklusok
16. Math
17. Tömbök
18. Gyakorlás

A félév beosztása

- 13 oktatási hét
- 4 hiányzás lehetséges, az 5. esetén nem teljesített eredmény
- Gyakorlati ZH a 7. és 12. héten (2*50 pont)
- Pótlási vagy javítási alkalom a 13. és 14. héten
- Osztályozás a TVSZ szerint
- Javasolt a sololearn.com Introduction to C# kurzus elvégzése

Bevezetés

C# története

- A C#-ot a Microsoft fejlesztette 2000 környékén
- C++ és a Java inspirálta
- Cél: biztonságos és objektumorientált nyelv megalkotása

Miért tanuljuk?

- Alkalmazások: Windows asztali appok (WPF)
- Web: ASP.NET Core szerveroldai fejlesztés
- Játékok: Unity engine (mobil és PC játékok nagy része)
- Vállalati szoftverek (Azure Cloud)

Első program

```
1 namespace prog2
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             Console.WriteLine("Hello, C#!");
8         }
9     }
10 }
```

Fontos információk

- Fontos, a C# case-sensitive nyelv!
- Az utasítások pontosvesszővel (;) zárulnak!
- A félév során a Visual Studio 2026 környezetet fogjuk használni
- .Net10 szükséges (géptermekben ez van telepítve)

Kiírás és bekérés

Kiíratás

- C++-ban: cout « "Valami";
- C#-ban: Console.Write();
- Új sorral: Console.WriteLine();

Bekérés

- C++-ban: cin, getline, cin.getline
- C#-ban: Console.ReadLine();

Azonosítók

Aspektus	Leírás
Megengedett karakterek	Csak alfanumerikus karakterek és számok, valamint alulvonás ([A-Z], [a-z], [0-9], _)
Kezdő karakter	Számmal nem kezdődhet
Nincs szóköz	A változók nem tartalmazhatnak szóközt
Kulcsszavak	A változók nem lehetnek kulcsszavak, kivéve a '@' előtaggal
Unicode támogatás	A C# változók támogatják a Unicode karaktereket
Case-Sensitive	A C# case-sensitive!
Hossz korlát	A változók nem lehetnek hosszabbak 512 karakternél
Nem lehet dupla aláhúzás	Kettős aláhúzást nem lehet a változók nevében

táblázat: Azonosítók elnevezése

Változók

- A felhasználó által megadott új értékeket a RAM tárolja
- A visszakeresés és manipulálhatóságra hozták létre a **változókat**
- Kétféle szintaxis:
  - *típus* változóNeve = érték;
  - *típus* változóNeve;
- típus: Meghatározza a változóban tárolt érték típusát.
- változóNeve: Érvényes azonosítónak kell lennie.
- érték: Ez a változóban tárolt tényleges érték.
- Deklaráció és inicializáció: int y = 7;
- Deklaráció: int x;
- Inicializálás: x = 5;
- Inicializálás típusai:
  - Fordítási idejű
  - Futási idejű

Változók típusai

Név	Értékhalmaz	Helyfoglalás (bit)
bool - logikai	true vagy false	1
char - karakter	1 db karakter	16
string - karakterlánc	U	0-2^{32} *db karakter*
byte, sbyte	0-255, -128-127	8
ushort, short	0-65535, -32708-32767	16
uint, int	0-4,2 * 10^9, $-2,1 * 10^9 - 2,1 * 10^9$	32
ulong, long	0-18 * 10^{18}, $-9 * 10^{18} - 9 * 10^{18}$	64
Lebegőpontos számok		32-128

táblázat: Változók típusai

Fordítás idejű inicializálás

```
1 namespace prog2
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             int ev = 2026;
8             Console.WriteLine("Aktuális év: " + ev);
9         }
10    }
11 }
```

Futás idejű inicializálás

```
1 namespace prog2
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             Console.Write("Add meg a neved: ");
8             string nev = Console.ReadLine();
9             Console.WriteLine("Üdvözöllek, " + nev);
10        }
11    }
12 }
```

Változók maximum és minimum értékei

```
1 namespace prog2
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             Console.WriteLine("Max of int: {0}", int.MaxValue);
8             Console.WriteLine("Min of int: {0}", int.MinValue);
9             Console.WriteLine("Max of double: {0}", double.MaxValue);
10            Console.WriteLine("Min of double: {0}", double.MinValue);
11        }
12    }
13 }
```

Konvertálás, castolás

```
1 namespace prog2
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             int szam1 = 5;
8             int szam2 = 120;
9             double maradekNelkul = szam1 / szam2; //0-t fogunk kapni
10            double maradekCastolas = (double)szam1 / szam2;
11            double maradekConvert = Convert.ToDouble(szam1) / szam2;
12            Console.WriteLine($"Maradék nélküli eset: {maradekNelkul}");
13            Console.WriteLine($"Castolás esetén: {maradekCastolas}");
14            Console.WriteLine($"Konvertálás esetén: {maradekConvert}");
15        }
16    }
17 }
```

Parse, TryParse

- Mindkét függvény csak **string** típusból tud átalakítást végezni
- Parse esetén, ha pl. betű kerül a számok közé, akkor hibát kapunk
- TryParse estén a visszatérési érték *bool* típusú
- Szintaktikája: típus.TryParse(bemenetiString, out típusnakMegfelelőVáltozó);

Példa a használatra

```
1 namespace prog2
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             string szoveg = "2026";
8             int ev1 = int.Parse(szoveg);
9             int ev2;
10            if (int.TryParse(szoveg, out ev2))
11                Console.WriteLine("Siker");
12            else
13                Console.WriteLine("Nem sikerült");
14            Console.WriteLine($"Parse: {ev1}, TryParse: {ev2}");
15        }
16    }
17 }
```

Elágazások - if-else

- Minimum 1 ág (if), de lehet több is (if-else if-else)
- Egymásba ágyazhatóak szinte bármennyiszer
- Egy ilyen szerkezetben mindig csak 1 ág adhat igazat, tehát az igaz, és a hamis ág nem futhat le egyszerre!
- Relációs és egyenlőségi operátorok használhatóak
- Ezek lehetnek: <, >, <=, >=, ==, !=
- Összetett feltételeket adhatunk meg az AND (&&) és az OR (||) segítségével

```
1    if (feltétel1) {
2        utasítás1;
3    } [else if (feltétel2) {
4        utasítás2;
5    } else {
6        utasítás3;
7    }]
```

Feladat

Készítsük el az aktuális TVSZ szerinti értékelést. Kérjük be a 2 ZH eredményét, és számítsuk ki az elért eredményt!

- 0-49%: Elégtelen - 1
- 50-60%: Elégséges - 2
- 61-75%: Közepes - 3
- 76-85%: Jó - 4
- 86-100%: Jeles - 5

Megoldás

```
1 namespace prog2
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             int zh1, zh2;
8             Console.Write("Add meg az 1. zh eredményét: ");
9             zh1 = int.Parse(Console.ReadLine());
10            Console.Write("Add meg a 2. zh eredményét: ");
11            zh2 = int.Parse(Console.ReadLine());
12
13            if (zh1 + zh2 > 85)
14                Console.WriteLine("Jeles - 5");
15            else if (zh1 + zh2 > 75)
16                Console.WriteLine("Jó - 4");
```

Megoldás

```
17                  else if (zh1 + zh2 > 60)
18                      Console.WriteLine("Közepes - 3");
19                  else if (zh1 + zh2 > 49)
20                      Console.WriteLine("Elégséges - 2");
21                  else
22                      Console.WriteLine("Elégtelen - 1");
23              }
24          }
25      }
```

Elágazások - switch-case

- Olyan elágazás, amelyben szinte végtelen ág lehetséges
- Csak egyenlőséget (egyezőséget) tud vizsgálni!
- break; utasítás szükséges!
- deafult ág, amely akkor fut le, ha minden korábbi ág hamis értéket adott
- Ez is egymásba ágyazható
- Többnyire akkor használjuk, ha az értékek kötöttek (pl: dobókocka)

```
1 switch (feltétel) {
2     case érték1:
3         utasítás1; break;
4     //...
5     default:
6         utasításN; break;
7 }
```

Feladat

Kérjen be egy dobókocka dobást 1 és 6 között! Ha nem ennek megfelelően adja meg a felhasználó az értéket, írjon ki hibaüzenetet!

Megoldás

```
1 namespace prog2
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             int dobas;
8             Console.Write("Adja meg a dobott számot (1-6): ");
9             dobas = int.Parse(Console.ReadLine());
10
11            switch (dobas)
12            {
13                case 1:
14                    Console.WriteLine("A dobott érték: 1"); break;
15                case 2:
16                    Console.WriteLine("A dobott érték: 2"); break;
17                case 3:
18                    Console.WriteLine("A dobott érték: 3"); break;
```

Megoldás

```
19                  case 4:
20                      Console.WriteLine("A dobott érték: 4"); break;
21                  case 5:
22                      Console.WriteLine("A dobott érték: 5"); break;
23                  case 6:
24                      Console.WriteLine("A dobott érték: 6"); break;
25                  default:
26                      Console.WriteLine("Hibás érték!"); break;
27              }
28          }
29      }
30  }
```

Ciklusok - elöltesztelő ciklus

- Először megvizsgálja a feltételt, utána végrehajta az utasítást
- Szélsőséges esetben egyszer sem fut le
- Általában létrehozunk egy változót, amit vizsgálhatunk
- Szintaxisa:

```
1 while (feltétel)
2 {
3     utasítás;
4 }
```

Ciklusok - hátultesztelő ciklus

- Először végrehajta az utasítást, utána megvizsgálja a feltételt
- Egyszer mindig lefut
- Általában ellenőrzött adatbekéréskor alkalmazzuk
- Addig fut, amíg meg nem felel a bekért érték a feltételnek
- Szintaxisa:

```
1 do
2 {
3     utasítás;
4 } while (feltétel);
```

Ciklusok - for ciklus

- Számlálóvezérelt ciklus
- Létrehozunk egy számlálót, amelynek az értékét minden iteráció után változtatjuk
- Addig fut, amíg a feltételünk igaz
- Testvére a **foreach**, amelyet tömbök esetén fogunk használni
- Szintaxisa:

```
1 for (int i = 0; i < valami; i++)
2 {
3     utasítás;
4 }
```

Math

- A *Math* osztály függvényei a matematikában megszokott képletek helyettesítésére szolgál
- Általános alakja: Math.függvény(érték(ek));
- A Math alap függvényei:
  - Abs(): az abszolútértéket adja vissza
  - Ceiling(): a megadott értéknél nem kisebb egész számot adja vissza
  - Floor(): a megadott értéknél nem nagyobb egész számot adja vissza
  - Round(): a megadott érték kerekített értéke, megadható a mértéke
  - Sin(): szinusz értéket adja vissza
  - Cos(): koszinusz értéket adja vissza
  - Tan(): tangens értéket adja vissza
  - Log(): természetes logaritmust adja vissza
  - Log10(): 10-es alapú logaritmust adja vissza

Math

- A Math alap függvényei (folytatás):
  - Min(): a két megadott érték közül a kisebbet adja vissza
  - Max(): a két megadott érték közül a nagyobbat adja vissza
  - Pow(): hatványozás, az első tag a hatványalap, a második a kitevő
  - Sqrt(): négyzetgyök értéket adja vissza
  - PI: a *pi* konstans értéke
  - E: az *e* konstans értéke
- Ezek használatára lásd a *01.29.cs* fájlt!

Tömbök

- Ezek összetett adatszerkezetek
- Jelenleg csak az 1 dimenziós tömböket nézzük meg
- Hasonlít a változóra, csak egy változóhoz több értéket is kapcsol
- Az indexelés a 0-tól indul!
- Alapértelmezetten 0-val vagy *null* értékkel töltődik fel
- n méretű tömb legnagyobb indexe: *n-1*
- Szintaxisa:

```
típus[] elnevezés = new típus[méret];
```

Feladat

Tárolja el egy csoport pontszámát (egész) egy tömbben. Számítsa ki az átlagát, valamint írassa ki a legkisebb és legnagyobb értékeket keresés tételével.

Megoldás

```
1 namespace prog2
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             int[] pontok = new int[] { 12, 11, 38, 60, 78, 15, 60, 100,
8             20, 12, 6, 97, 81, 45, 2, 64 };
9             double atlag = 0;
10            int legkisebb = pontok[0], legnagyobb = pontok[0];
11            for (int i = 0; i < pontok.Length; i++)
12            {
13                atlag += pontok[i];
14                if (pontok[i] < legkisebb)
15                    legkisebb = pontok[i];
16                if (pontok[i] > legnagyobb)
17                    legnagyobb = pontok[i];
18            }
```

Megoldás

```
18                Console.WriteLine("A csoport pontszámai:");
19                foreach (int i in pontok)
20                {
21                    Console.WriteLine(i);
22                }
23                Console.WriteLine($"A csoport átlaga: {atlag / pontok.Length}");
24                Console.WriteLine($"A legkisebb pontszám: {legkisebb}");
25                Console.WriteLine($"A legnagyobb pontszám: {legnagyobb}");
26            }
27        }
28    }
```

Gyakorlás

- Kérjen be ellenőrzötten egy pozitív egész számot, és számítsa ki a faktoriálisát!
- Készítse el a Fibonacci sorozatot kiíró programot!
- Kérjen be egy számot, majd ennek megfelelő méretben hozzon létre egy tömböt, és töltse fel ellenőrzötten 0 és 100 közötti számokkal!
- Szimulálja a „Buggy egy, jobbra megy" játékot. A körben ülő játékosok sorban mondanak egymást követő egész számokat. Amikor 7-re végződő, vagy 7-tel osztható szám jönne, helyette azt mondja a játékos: „Buggy (hibás)" és ekkor a körbejárás megfordul. Jobbra kezdünk. Gombnyomásra léptessen! A szimulátor eredménye és kimenete: (1 j 2 j 3 j 4 j 5 j 6 j Buggy 8 b 9 b 10 b 11 b 12 b 13 b Buggy 15 j 16 j Buggy 18 b 19 b)